

n-document-header >}}

## Summary

GitLab codebase has grown a lot since the first commit made in 2011. We've been able to implement many features that got adopted by millions of users. There is a demand for more features, but there is also an opportunity of a paradigm change: instead of delivering features that cover specific use-cases, we can start building a platform that our users will be able to extend with automation as they see fit. We can build a flexible and generic DevSecOps solution that will integrate with external and internal workflows using a robust eventing system.

In this design document we propose to add a few additional layers of abstraction to make it possible to:

1. Design a notion of events hierarchy that encodes their origin and schema.
1. Publish events from within the application code using Publishers.
1. Intercept and transform events from external sources using Gateways.
1. Subscribe to internal / external events using Subscribers.
1. Hide queueing and processing implementation details behind an abstraction.

This will allow us to transform GitLab into a generic automation tooling, but will also reduce the complexity of existing events-like features:

1. Webhooks
1. Audit Events
1. GitLab CI Events
1. Package Events
1. GraphQL Events

## Goals

Build required abstractions and their implementation needed to better manage internally and externally published events.

## Challenges

1. There is no solution allowing users to build subscribers and publishers.
1. There is no solution for managing subscriptions outside of the Ruby code.
1. There are many events-like features inside GitLab not using common abstractions.
1. Our current eventing solution `Gitlab::EventStore` is tightly coupled with Sidekiq.
1. There is no unified and resilient way to subscribe to externally published events.
1. Payloads associated with events differ a lot, similarly to how we define schemas.
1. Not all events are strongly typed, there is no solution to manage their hierarchy.
1. Events are not being versioned, it is easy to break schema contracts.
1. We want to build more features based on events, but because of missing abstractions the value we could get from the implementations is limited.

## Proposal

## Publishers

Publishing events from within our Rails codebase is an important piece of the proposed architecture. Events should be strongly typed, ideally using Ruby classes.

For example, we could emit events in the following way:

```
ruby
include Gitlab::Events::Emittable

emit Gitlab::Events::Package::Published.new(package)
```

- Publishing events should be a non-blocking, and near zero-cost operation.
- Publishing events should take their origin and identity into the account.
- Publishing events should build their payload based on their lineage.
- emit can be a syntactic sugar over mechanism used in GitLab::EventStore.

## Subscribers

Subscribers will allow application developers to subscribe to arbitrary events, published internally or externally. Subscribers could also allow application developers to build subscription mechanisms that could be used by our users to, for example, subscribe to project events to trigger pipelines.

Events that subscribers will subscribe to will become contracts, hence we should version them or use backwards-and-forward compatible solution (like Protobuf).

## Gateways

Gateways can be used to intercept internal and external events and change their type, augment lineage and transform their payloads.

Gateways can be used, for example, to implement sink endpoints to intercept Cloud Events, wrap into an internally used Ruby classes and allow developers / users to subscribe to them.

We also may be able to implement cross-Cell communication through a generic events bus implemented using Gateways.

There are also ideas around cross-instance communication to improve how GitLab can coordinate complex deployments that involve multiple instances.

## Processing

Today in order to queue events, we either use PostgreSQL or Sidekiq. Both mechanisms are being used interchangeably and are tightly coupled with existing solution.

The main purpose of building an abstraction for queuing and processing is to be able to switch to a different queuing backend when needed. For example, we could queue some of the events on Google Pub/Sub, and send those through a dedicated Gateway on their way back to the application.

## Observability

In order to understand interactions between events, publishers and subscribers we may need to deliver a proper instrumentation via OpenTelemetry. This will allow us to visualize these interactions with Distributed Tracing Backends.

---

## SECTION: engineering/architecture/design-documents/gitlab\_housekeeper/\_index.md

{{< engineering/design-document-header >}}

## Summary

This blueprint documents the philosophy behind the "GitLab Housekeeper" gem which was introduced in <<https://gitlab.com/gitlab-org/gitlab/-/mergerequests/139492>> and has already been used to create many merge requests.

The tool should be used to save developers from mundane repetitive tasks that can be automated. The tool is scoped to any task where a developer needs to create a straightforward merge request and is known ahead of time.

This tool should be useful for at least the following kinds of mundane MRs we create:

1. Remove a feature flag after X date
1. Remove an unused index where the unused index is identified by some automation
1. Remove an ignorecolumn after X date (part of renaming/removing columns multi-step procedure)
1. Populate sharding keys for organizations/cells on tables that are missing a sharding key

## Motivation

We've observed there are many cases where developers are doing a lot of manual work for tasks that are entirely predictable and automatable. Often these manual tasks are done after waiting some known period of time. As such we usually create an issue and set the future milestone. Then in the future the developer remembers to followup on that issue and opens an MR to make the manual change.

The biggest examples we've seen lately are:

1. Feature flag removal: <<https://gitlab.com/groups/gitlab-org/-/epics/5325>>. We have many opportunities for automation with feature flags but this blueprint focuses on removing the feature flag after it's fully rolled out. A step that is often forgotten leading to growing technical debt.
1. Removing duplicated or unused indexes in Postgres: <<https://gitlab.com/gitlab-org/gitlab/-/issues/385701>>. For now we're developing automation that creates issues and assigns them to groups to follow up and manually open MRs to remove them. This blueprint would take it a step further and the automation would just create the MRs to remove them once we have identified them.
1. Removing out of date ignorecolumn references: <<https://docs.gitlab.com/ee/development/database/avoidingdowntimeinmigrations.html#removing-the-ignore-rule-release-m2>>  
. For now we leave a note in our code telling us the date it needs to be removed and often create an issue as a reminder. This blueprint proposes that automation just reads this note and opens the MR to remove it after the date.
1. Adding and backfilling sharding keys for organizations for Cells: <<https://gitlab.com/gitlab-org/gitlab/-/mergerequests/133796>>. The cells architecture depends on all tables having a sharding key that is attributed to an organization. We will need to backfill this for ~300 tables. Much of this will be repetitive and mundane work that we can automate provided that groups just identify what the name of the sharding key should be and how we will backfill it. As such we can automate the creation of MRs that guess the sharding key and owning groups can check and correct those MRs. Then we can automate the MR creation for adding the columns and backfilling the data. Some kind of automation like this will be necessary to finish this work in a reasonable timeframe.

## Goals

1. Identify the common tasks that take development time and automate them.
1. Focus on MR creation rather than issue creation as MRs are the results we want and issues are a process for reminding us to get those results.
1. Improve developer job satisfaction by knowing that automation is doing the busy work while we get to do the challenging and creative work.
1. Developers should be encouraged to contribute to the automation framework when they see a pattern rather than documenting the manual work for future developers to do it again.
1. Automation MRs should be very easily identified and reviewed and merged much more quickly than other MRs. If our automation MRs cause too much effort for reviewers we maybe will outweigh the benefits. This might mean that some automations get disabled when they are just noisy.

## Solution

The  
GitLab Housekeeper gem  
should be used to automate creation of mundane merge requests.

Using this tool reflects our bias for action subvalue. As such, developers should preference contributing a new keep over the following:

1. Documenting a process that involves creating several merge requests over a period of time
1. Setting up periodic reminders for developers (in Slack or issues) to create some merge request

The keeps may sometimes take more work to implement than documentation or reminders so judgement should be used to assess the likely time savings from using automation. The gitlab-housekeeper gem will evolve over time with many utilities that make it simpler to contribute new keeps and it is expected that over time the cost to implementing a keep should be small enough that we will mostly prefer this whenever developers need to do a repeatable task more than a few times.

## Design and implementation details

The key details for this architecture is:

1. The design of this tool is like a combination of rubocop -a and Renovate bot. It extends on rubocop -a to understand when things need to be removed after certain deadlines as well as creating a steady stream of manageable merge requests for the reviewer rather than leaving those decisions to the developer. Like the renovate bot it attempts to create MRs periodically and assign them to the right people to review.
1. The keeps live in the GitLab repo which means that there are no dependencies to update and the keeps can use code inside the GitLab codebase.
1. The script can be run locally by a developer or can be run periodically in some automated way.
1. The keeps are able to use any data sources (eg. local code, Prometheus, Postgres database archive, logs) needed to determine whether and how to make the change.

---

## SECTION: engineering/architecture/design-documents/gitlab\_messaging\_layer/\_index.md

{{< engineering/design-document-header >}}

## Summary

This document proposes design, architecture and a rollout roadmap for adopting & using a messaging layer to support data messaging & queuing needs at GitLab scale.

From some of our recent initiatives such as building the Data Insights Platform or Project Siphon, it has become evident that we need a scalable & reliable queueing system within our

technology stack to be able to ingest & process large amounts of data. Having gone through multiple discussions around this, we have narrowed down our choices to using NATS as a solution to these needs - as explained later in this document.

## Motivation

A primary driver for having a scalable messaging layer within our tech-stack is building our ability to ingest large amounts of data, especially analytical or monitoring data. While it is totally possible to persist data directly into our databases, it's riddled with scalability challenges. For example, when ingesting data into ClickHouse, the database given its architecture - performs much better when ingesting large batches of data across fewer writes than ingesting a large number of small writes. Some past context around our experience with this.

Another key requirement is to be able to process incoming data before it lands within a database. At minimum, we need the ability to perform operations such as:

- dynamically enrich incoming data with metadata from other systems & data catalog,
- ensure ingested data does not contain sensitive information and anonymize, pseudonymize, or redact data as needed,
- batch data outside of main storage while certain (logical) conditions are met,
- fan-out ingested/processed data to multiple destinations, etc.

From an architectural perspective, having a data buffer available upstream to our persistent stores would help alleviate resource pressure downstream by absorbing large spikes in ingested data, which might happen quite frequently as we continue to generate and/or gather more data. We outline other significant benefits of building such an abstraction later in this document.

Considering our forward-looking use-cases, we also need such a system to be consistently available across all our deployment-models for GitLab instances, i.e. GitLab.com, Cells, Dedicated or Self-Managed. This would ensure we consolidate how we deal with any data stream generated across the product. With such a large deployment surface, it's important we minimize all distribution & operational complexities of running such a system while ensuring its reliable, scalable and capable of delivering performance at GitLab scale.

As elucidated later in this document, NATS stands out given its minimal footprint, ease of distribution and its ability to both be embeddable within the product and scale out as a cluster when needed.

## Goals

- Establish scalable & reliable data queueing infrastructure for a few first adopters: Siphon, Data Insights Platform.
- Provide authenticated & authorized access to all ingested data into the messaging layer.
- Provide necessary documentation to allow developers to interact with the messaging layer.
- Integrate messaging layer implementation with existing GitLab infrastructure to enable the aforementioned use-cases.

## Non-goals

- Not aim to build a general-purpose event-bus for all our queueing/eventing needs just yet. Rather, the blueprint aims to lay the foundation for a future comprehensive event-bus with design decisions that should preserve compatibility with broader applications or use-cases looking forward.
- Not cover application-specific implementation details within the context of a messaging layer.

## Proposal

The core of this proposal is to establish a foundational messaging piece within our tech-stack and build out necessary infrastructure well-integrated with GitLab installation(s).

For this first iteration, we do not expect to have all GitLab services or applications interacting with this messaging layer directly. The only planned usage right now is the following:

- For Siphon to buffer Postgres replication events before landing them in ClickHouse.
- Applications sending Snowplow-instrumented events to Data Insights Platform via event instrumentation layer which are dynamically enriched and landed into ClickHouse and AWS S3.

## Looking forward

Once the aforementioned messaging layer is generally available, we aim to position it as the data queueing backbone for a more general-purpose events-based data platform within the product. Its existence helps ensure reliability and scalability for the various cross-platform data-related features at GitLab.

## Benefits of building an events-based platform within GitLab

Having a centralized events-based data platform within the product helps improve GitLab's logical architecture and its ability to scale well with time. As we build & continue to adopt such an architecture, the following key benefits come to mind:

- Enables loose-coupling between data producers and consumers across GitLab allowing them to scale independently of each other.
- Encourages asynchronous communication patterns between participating systems improving their scalability with increasing traffic volumes.
- Makes our architecture extensible wherein new consumers of existing data can be added with trivial time & effort.
- Provides a centralized, common architecture for sharing data useful for integrations across different parts of the product. This also leads to building over time singular sources of truth for important data across the product bringing consistency to product data.

The following discussions also explain why building an events-based abstraction is important for GitLab's architecture at its current scale:

- Product Event Platform
- GitLab Events Platform
- GitLab Structured Events

Note, while the existence of a centralized data-sharing platform helps alleviate scalability & reliability concerns from other parts of our infrastructure, esp. databases, we will also need to iron out a few other concerns such as authorising clients, routing data efficiently and decoupling away from the monolith to ensure such a system proves valuable to our logical architecture.

### Identified use-cases

Following is a detailed set of use-cases that benefit from the existence of a centralized data platform within the product.

| Teams/areas                    | Use-cases                                                                                                                                                                                                                                                                                                                                         | Expected scale                                                                                                                                                                                                                             |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enterprise Data/Infrastructure | Logically replicating data out of Postgres.                                                                                                                                                                                                                                                                                                       | ~100GB of new data ingested from Postgres per hour. We'll also need to scale with Cells architecture.                                                                                                                                      |
| Platform Insights              | Ingesting & processing large amounts of analytics data in real-time, then persisting it into ClickHouse.                                                                                                                                                                                                                                          | Product Usage Data: 1200 events/sec (100M daily), ~15GB/hour, expected to increase with increase in instrumentation + 2.5x when pursuing event-level data collection from the customer's domain (about 300M events daily).                 |
| Machine Learning               | Extracting & processing events/features from GitLab data to create training/test datasets for ML models at scale.                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                            |
| Security & Compliance          | Extracting and actioning upon events, e.g., audit events, from GitLab data in near real-time. Audit events saved to database per day (excludes streaming only audit events) - ~0.6M records are created per day (audit event coverage has stalled due to load on Postgres, this is likely to increase if this migrates over to an event pipeline) | Streaming only audit events generated per day (this excludes audit events saved to DB) - ~35M streaming only events are created per day. Total estimates would be at ~40M events per day. See Kibana dashboard (internal).                 |
| Product                        | Generic Events Platform to asynchronously process data, events & tasks. Ingesting & processing external data via webhooks as a service.                                                                                                                                                                                                           |                                                                                                                                                                                                                                            |
| Product                        | Implementing real-time analytics features on top of an analytical database (ClickHouse). (primarily the Optimize team would be involved, other product teams might also contribute)                                                                                                                                                               | Data volume: Similar or less than what we observe with the PostgreSQL databases. Depends on how many tables we replicate (Siphon) to ClickHouse. Enqueued event count: significantly lower as we're batching the CDC events into packages. |
| Plan                           | JIRA Compete Strategy                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                            |

### Considered alternatives

As a potential solution to the aforementioned use-cases, a few popular backends were considered throughout our discussions. The following comparison matrix helps assess different possible backends for messaging needs within GitLab.

Note, we considered the following four deployment targets (for GitLab) when assessing potential solutions:

- GitLab.com SaaS: Multi-tenant instance.
- GitLab Dedicated: Single-tenant, dedicated resources per instance.
- Self-Managed on-cloud: For this environment, we assume a GCP-environment to prefer using GCP resources, an AWS-environment to prefer using AWS resources, etc. accounting for deployment-



homogeneity.

- Self-Managed on-premise: For this environment, we assume all environments to be air-gapped so as to not have to make other assumptions on behalf of our customers.

| Dimension / Solution | Apache Kafka | RabbitMQ | Google PubSub | AWS Kinesis | NATS |  
Our preferred solution for the given dimension(s) |

| ---- | ---- | ---- | ---- | ---- | ---- | ---- |

| Distribution: Assessing packaging and distribution complexity of the system. | Available as a JVM-based application, pre-packaged for most environments we intend to run it in. <br><br> Score: 6/10 | Available pre-packaged for most environments we intend to run it in, though having support for Erlang as a runtime makes distribution non-trivial compared to e.g. Go or Java. <br><br> Score: 5/10 | Externally managed cloud solution, available on Google Cloud Project. <br><br> Score: 10/10 | Externally managed cloud solution, available on Amazon Web Services. It scores slightly less than GCP considering our tight integrations on GCP for hosting .com SaaS. <br><br> Score: 9/10 | Available as a compiled, lightweight, single-binary developed in Golang. NATS can be packaged and deployed in all environments we support running GitLab instances in. <br><br> Score: 8/10 | NATS or cloud-managed services subject to self-hosting challenges. |

| Operational Complexity: Considering management of this solution in: <br> GitLab.com SaaS (multitenant). <br> GitLab Dedicated (single tenant). <br> Self-Managed (SM) on Cloud. <br> Self-Managed (SM) on-premise. | Kafka can be non-trivial to operate, generally speaking. <br><br> .com SaaS: Should be non-trivial to add & support with a very large storage footprint. · <br><br> Dedicated: Should be non-trivial to add & support, though storage footprint might be manageable. · <br><br> SM on Cloud: Should be trivial to add & support though it's highly cost-intensive even on smaller reference architectures. See \[1], \[2] for past discussions. <br><br> SM on-prem: High upfront capex & opex unless already available within the environment. · <br><br> Score: 5/10 | RabbitMQ can be non-trivial to operate, generally speaking. <br><br> .com SaaS: Should be non-trivial to add & support with a reasonably large large storage footprint. · <br><br> Dedicated: Should be non-trivial to add & support, though storage footprint should be manageable. · <br><br> SM on Cloud: Trivial to add support, though highly cost-intensive even with smaller reference architectures. · <br><br> SM on-prem: High upfront capex & opex unless already available within the environment. · <br><br> Score: 4/10 | Google PubSub comes with near-zero operational complexity being an externally managed cloud solution. <br><br> .com SaaS: Should be trivial to add & support. · <br><br> Dedicated: Should be trivial to add & support should we need to build Dedicated environments in GCP. · <br><br> SM on Cloud: Should be trivial to add & support. · <br><br> SM on-prem: Discarding the possibility of use assuming the environment to be air-gapped. · <br><br> Score: 8/10 | Amazon Kinesis comes with near-zero operational complexity being an externally managed cloud solution. <br><br> .com SaaS: Should be trivial to add & support. · <br><br> Dedicated: Should be trivial to add & support considering Dedicated environments already exist on AWS. · <br><br> SM on Cloud: Should be trivial to add & support. · <br><br> SM on-prem: Discarding the possibility of use assuming the environment to be air-gapped. · <br><br> It might also be worth noting that Dedicated uses Kinesis for log aggregation and frequently experiences performance problems with it. See these issues for more context. <br><br> Score: 7/10 | NATS is reasonably trivial to operate, given its architecture. <br><br> .com SaaS: Should be trivial to add & support, storage footprint can be managed on a per use-case basis. · <br><br> Dedicated: Should be trivial to add & support, storage footprint can be managed on a per use-case basis. · <br><br> SM on Cloud: Support not available on major hyperclouds but available via Synadia Cloud. · <br><br> SM on-prem: Should be non-trivial to add & support, storage footprint can be managed on a per use-case basis. · <br><br> Score: 8/10 | NATS or cloud-managed services subject to

self-hosting challenges. Note, for most self-managed on-premise installations, NATS stands out as a potential solution. |

| Availability as a Managed Service: Do AWS, Google Cloud or Azure provide a managed service? | AWS-managed Kafka (MSK) GCP Managed service for Kafka Azure Messaging Services OR Hosted Kafka on Azure, supported by Canonical. <br><br> Score: 10/10 | Amazon MQ Hosted RabbitMQ on Azure supported by CloudAMQP. <br><br> Score: 6/10 | Managed service. <br><br> Score: 10/10 | Managed service. <br><br> Score: 9/10 | Not available on the major hyperclouds directly but it is available as a managed cloud solution via Synadia Cloud. <br><br> Score: 6/10 | Google PubSub or AWS Kinesis subject to which environment we're deploying our applications to. <br><br> Integration with Synadia Cloud is pending exploration. |

| Consistency/Maturity: Jepsen reports, other potential concerns with consistency? | Jepsen testing has been done for Kafka, no consistency concerns outside of normal distributed systems exist. <br><br> Score: 9/10 | Jepsen testing has been done for RMQ, no consistency concerns outside of normal distributed systems exist. <br><br> Score: 9/10 | No concerns. <br><br> Score: 10/10 | No concerns. <br><br> Score: 10/10 | Jepsen tests have not been performed yet, though NATS has had considerable production presence across its set of adopters. <br><br> Score: 8/10 | No outright blockers against using any of the explored systems. |

| Cloud Native Support: How complex is it to run the service in Kubernetes? | Yes, popularly deployed via Kubernetes Operators such as Strimzi. <br><br> Score: 5/10 | Yes, also deployable via the Kubernetes operator officially supported by the RMQ team. <br><br> Score: 5/10 | Managed service, integration only. <br><br> Score: 10/10 | Managed service, integration only. <br><br> Score: 10/10 | Yes, deployable via Helm charts. <br><br> Score: 9/10 | NATS or cloud-managed services subject to self-hosting challenges. |

| License Compatibility: Is the system being considered license-compatible with GitLab? As part of this we also considered the likelihood of license changes. | Kafka is licensed as Apache Version 2.0, available as open-source. <br><br> Score: 10/10 | RabbitMQ is dual-licensed under the Apache License 2.0 and the Mozilla Public License 2. <br> We have the freedom to use and modify RabbitMQ however we want. <br><br> Score: 10/10 | Managed, commercially licensed. <br><br> Score: 10/10 | Managed, commercially-licensed. <br><br> Score: 10/10 | NATS is licensed as Apache Version 2.0. Originally developed by Synadia, has now been donated to CNCF, and is currently under incubation. <br><br> Score: 10/10 | No red flags, most systems are appropriately licensed. |

| Compatibility with our technology stack: Is the technology stack compatible with our experience? | JVM-based system. <br><br> Score: 2/10 | RabbitMQ can potentially run on any platform that provides a supported Erlang version, from multi-core nodes and cloud-based deployments to embedded systems. <br><br> Score: 1/10 | Highly compatible with our technology stack, especially in how we approach systems and integrations at GitLab. <br><br> Score: 10/10 | Highly compatible with our technology stack, especially in how we approach systems and integrations at GitLab. Scores slightly less than PubSub given our existing presence within GCP. <br><br> Score: 9/10 | Reasonably compatible with our tech-stack, with NATS developed in Golang. Given it compiles to a single binary, it can run natively on host machines across our deployment surface with trivial effort. <br><br> Score: 9/10 | NATS or cloud-managed services subject to self-hosting challenges. |

| Client/Language support: How good/bad is client/language support for the given system, most notably Golang, Ruby and Python? | Widely supported across major languages, with other languages supported via librdkafka wrappers. <br><br> Golang: · <br> Ruby: · <br> Python: · <br><br> Score: 9/10 | Widely supported across languages. <br><br> Golang: · <br> Ruby: · <br> Python: · <br><br> Score: 10/10 | Has support for major languages via officially supported clients. <br><br> Golang: · <br> Ruby: · <br> Python: · <br><br> Score: 10/10 | Provides Kinesis Client Library (KCL) as a standalone Java software library designed to

simplify the process of consuming and processing data from Amazon Kinesis Data Streams. We need to use language specific SDKs or KCL wrappers for other languages. <br><br> Golang: · <br> Ruby: · <br> Python: · <br><br> Score: 8/10 | Widely supported across languages <br><br> Golang: · <br> Ruby: · <br> Python: · <br><br> Score: 10/10 | No clear winners here. All considered solutions have decent client-support for the three languages we need support in, with varying levels of support for other languages. |

| Dependencies: How much supply-chain risk do we introduce by including this dependency? How big/deep is the dependency graph for this project? | Needs Zookeeper or KRaft for cluster co-ordination, in addition to running Kafka brokers themselves. ZK can run natively on underlying hosts, while KRaft runs as an internal part of Kafka (broker) process. <br><br> Score: 5/10 | Needs the support for an Erlang runtime, code-developed in Erlang. The dependency-graph is mostly unfamiliar technology for our environments. <br><br> Score: 4/10 | Managed, closed-source systems. We do not actively manage the dependency graph on this system. <br><br> Score: 9/10 | Managed, closed-source systems. We do not actively manage the dependency graph on this system. <br><br> Score: 9/10 | Single Golang binary with no external dependencies, and runs natively on underlying hosts. <br><br> Score: 8/10 | NATS or cloud-managed services subject to self-hosting challenges. |

| FIPS Compliance: Assess FIPS 140-3 readiness and/or AWS GovCloud managed service options. | Kafka itself doesn't support FIPS compliance but can be configured on runtime with a validated JDK/OpenSSL setup. <br><br> Score: 6/10 | Similar to Kafka, RabbitMQ must be run with a FIPS-compliant OpenSSL library and/or dependencies. <br><br> Score: 6/10 | Yes, implicit support via Google's FIPS-compliant cryptography. <br><br> Score: 10/10 | Yes, deployments must be secured behind AWS FIPS endpoints. <br><br> Score: 9/10 | Go's crypto-implementation isn't FIPS ready but NATS deployments can be secured via a FIPS-compliant TLS implementation using OpenSSL. <br><br> Chainguard, a GitLab partner, does provide a FIPS image for NATS. <br><br> Score: 6/10 | NATS or cloud-managed services subject to self-hosting challenges. |

| Exploits and CVEs: How many critical CVEs has the product experienced over the past 2 years?  
| | | | |

| Community Adoption: How widely adopted is the system? Contributions from multiple organisations, etc. | Widely adopted. <br><br> Score: 10/10 | Widely adopted. <br><br> Score: 9/10 | Widely adopted. <br><br> Score: 10/10 | Widely adopted. <br><br> Score: 10/10 | Widely adopted. <br><br> Score: 9/10 | No clear winners here. All considered solutions have decent footprints subject to the different dimensions applicable to different users. |

| Operational costs: How expensive is it to adopt/run the system esp. at our scale? <br><br> Note: See detailed analysis of costs and its breakdown with used reference architectures and underlying sizing later in this document. | When compared to NATS or RabbitMQ, Kafka can be more resource intensive both in compute and storage given its replication overheads. It'll consume more compute nodes if we use Zookeeper for cluster coordination OR more vCPUs in the case of using KRaft which works on the same JVM as the broker process. <br><br> Score: 6/10 | RabbitMQ can be slightly more compute intensive as compared to NATS given its acknowledgement mechanism, wherein it's more suitable for more transactional streaming workloads. Operational costs roughly similar to NATS otherwise. <br><br> Score: 7/10 | Managed solution with a major share of costs coming from data transfer to & from the backing system. Additionally, having to retain data for longer costs more in GCP than AWS. <br><br> Score: 4/10 | Managed solution, with a major share of costs coming from data transfer to & from the backing system. <br><br> Score: 5/10 | Least compute intensive of the systems analysed, with a minimal footprint on storage as well. Scales horizontally with our reference architectures without incurring large overheads. <br><br> Score: 9/10 | NATS clearly stands out given its minimal footprint and lesser overheads assuming we can discount any self-hosting challenges. Any of the analysed managed services in the Cloud cost an order of magnitude more, especially at scale. |

| Net Score | 83 | 76 | 111 | 105 | 100 | \- |  
 | | Apache Kafka | RabbitMQ | Google PubSub | AWS Kinesis | NATS | - |

## Cost estimation & analysis

To ensure a comparable analysis for the different systems, we made some assumptions around how much data we need to host in each of the analysed systems.

For example, accounting for all Snowplow-instrumented data originating from .com SaaS, we estimate to generate 500GB of event data per day. If we then intend to retain this data for a week, we'll roughly accumulate 3.5TB data which will need to be hosted on the underlying infrastructure at any given point in time. It can be assumed that data lifecycle policies kick-in correctly and this remains our maximum storage footprint within the context of this example.

- Daily generated data: 500GB
- Retention: 7 days
- Maximum stored data:  $500\text{GB} \times 7 \text{ days} = 3.5\text{TB}$

The following is how all analysed backends fair with that amount of data.

| System / Components | Deployment Target | Compute | Storage | Network | Support/Operational | Total Estimate |
|---------------------|-------------------|---------|---------|---------|---------------------|----------------|
|---------------------|-------------------|---------|---------|---------|---------------------|----------------|

|       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|
| :---- | :---- | :---- | :---- | :---- | :---- | :---- |
|-------|-------|-------|-------|-------|-------|-------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |  |                                         |  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|-----------------------------------------|--|
| Apache Kafka                                                                                                                                                                                                                                                                                                                                                                                                                       |                                         | Estimated with the simplest possible architecture comprising 3 brokers, each using 4 vCPUs \+ 16GB RAM, and attached to 1TB standard persistent disks each. It's also expected we either colocate Zookeeper with Kafka brokers or use KRaft instead. When deploying Zookeeper nodes standalone, costs for those nodes will be additional. It's also expected no replication is set up on the topics/partitions being used. Additional replication for HA will increase the storage footprint n times, where n is the replication factor used.   .com SaaS   GCP n4-standard-4 (4 vCPUs and 16 GB RAM) @ \$0.1895 per hour. Monthly cost per VM: \$0.1895/hour \ 730 hours/month = \$138.34 Total for 3 VMs: 3 \ \$138.34 = \$415.02.   GCP Standard persistent disks @ \$0.04 per GB per month. Cost per 1 TB disk: 1,024 GB \ \$0.04/GB = \$40.96 Total for 3 disks: 3 \ \$40.96 = \$122.88.   GCP Subject to how the cluster is set up, the following network pricing rules apply. Intra-zone/Inter-zone data transfer costs may apply, i.e. VM-to-VM data transfer when the source and destination VM are in different zones of the same Google Cloud region. When zones are different, \$0.01 per GiB bandwidth used. We encourage both the server and the client to be in the same region to keep the communication efficient. Otherwise, inter-region data transfer costs apply for each GiB bandwidth used.     GCP Monthly Estimate: \$415.02 (compute) \+ \$122.88 (storage) = \$537.90 |  |                                         |  |
| AWS t3a.xlarge (4 vCPUs and 16 GB RAM) @ \$0.0832 per hour. Monthly cost per instance: \$0.0832/hour \ 730 hours/month = \$60.74 Total for 3 instances: 3 \ \$60.74 = \$182.22.   AWS General Purpose SSD gp2 volumes @ \$0.10 per GB per month. Cost per 1 TB volume: 1,024 GB \ \$0.10/GB = \$102.40 Total for 3 volumes: 3 \ \$102.40 = \$307.20.     AWS Monthly Estimate: \$182.22 (compute) \+ \$307.20 (storage) = \$489.42 |                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |  |                                         |  |
| Dedicated                                                                                                                                                                                                                                                                                                                                                                                                                          | (same as above)                         | (same as above)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |  | (same as above)                         |  |
| SM on-cloud                                                                                                                                                                                                                                                                                                                                                                                                                        | (same as above)                         | (same as above)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |  | (same as above)                         |  |
| SM on-prem                                                                                                                                                                                                                                                                                                                                                                                                                         | Estimated with reference architectures. | Estimated with reference architectures.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |  | Estimated with reference architectures. |  |
| RabbitMQ                                                                                                                                                                                                                                                                                                                                                                                                                           | .com SaaS                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |  |                                         |  |

|             |  |  |  |  |
|-------------|--|--|--|--|
| Dedicated   |  |  |  |  |
| SM on-cloud |  |  |  |  |
| SM on-prem  |  |  |  |  |

| NATS <br><br> Estimated with the simplest possible architecture comprising 3 NATS servers, each using 4 vCPUs \+ 16GB RAM, and attached to 1TB standard persistent disks each. No other dependencies are required. | .com SaaS | GCP n4-standard-4 (4 vCPUs and 16 GB RAM) @ \$0.1895 per hour <br><br> Monthly cost per VM: \$0.1895/hour \ 730 hours/month \= \$138.34 <br><br> Total for 3 VMs: 3 \ \$138.34 \= \$415.02. | GCP Standard persistent disks @ \$0.04 per GB per month. <br><br> Cost per 1 TB disk: 1,024 GB \ \$0.04/GB \= \$40.96 <br><br> Total for 3 disks: 3 \ \$40.96 \= \$122.88. | | | GCP Monthly Estimate: \$415.02 (compute) \+ \$122.88 (storage) \= \$537.90 |

| | | AWS t3a.xlarge (4 vCPUs and 16 GB RAM) @ \$0.0832 per hour. <br><br> Monthly cost per instance: \$0.0832/hour \ 730 hours/month \= \$60.74 <br><br> Total for 3 instances: 3 \ \$60.74 \= \$182.22. | AWS General Purpose SSD gp2 volumes @ \$0.10 per GB per month. <br><br> Cost per 1 TB volume: 1,024 GB \ \$0.10/GB \= \$102.40 <br><br> Total for 3 volumes: 3 \ \$102.40 \= \$307.20. | | | AWS Monthly Estimate: \$182.22 (compute) \+ \$307.20 (storage) \= \$489.42 |

|             |                                         |                                         |                                         |
|-------------|-----------------------------------------|-----------------------------------------|-----------------------------------------|
| Dedicated   | (same as above)                         | (same as above)                         | (same as above)                         |
| SM on-cloud | (same as above)                         | (same as above)                         | (same as above)                         |
| SM on-prem  | Estimated with reference architectures. | Estimated with reference architectures. | Estimated with reference architectures. |

| Google PubSub <br><br> Cost for PubSub has three main components: <br><br> Throughput costs for message publishing & delivery. <br> Data transfer costs associated with throughput that crosses a cloud zone or region boundary. <br> Storage costs. <br><br> Additionally: No zonal data transfer fees for usage. <br> No inbound data transfer fees are required when publishing from other cloud/private DCs. | .com SaaS | Message Delivery Basic SKU @ \$40 per TiB. | Storage @ \$0.27 per GiB-month. | Data transfer costs to be estimated via VPC network rates. Using standard tier pricing, \$0.045 per GiB/month. | | Given our capacity estimates: <br><br> Throughput: 500GB\30\0.04 \= \$600 per month. <br><br> Storage: 3.5TB\1024\0.27 \= \$968 per month. <br><br> Delivery: 500GB\30\3\0.045 \= \$1800 per month <br><br> Monthly estimate: \~ \$3400 |

|             |                                         |                                         |                                         |                                         |
|-------------|-----------------------------------------|-----------------------------------------|-----------------------------------------|-----------------------------------------|
| Dedicated   | (same as above)                         | (same as above)                         | (same as above)                         | (same as above)                         |
| SM on-cloud | (same as above)                         | (same as above)                         | (same as above)                         | (same as above)                         |
| SM on-prem  | Estimated with reference architectures. | Estimated with reference architectures. | Estimated with reference architectures. | Estimated with reference architectures. |

| Amazon Kinesis | .com SaaS | Shards: \$0.015 per shard-hour | Storage (>24h): \$0.023 per GB-month. | Incoming data: \$0.08 per GB Outgoing data: \$0.04 per GB | | Given our capacity estimates: <br><br> Ingestion: 500GB\30\0.08 \= \$1200 per month. <br><br> Delivery: 500GB\30\0.04 \= \$1800 per month <br><br> Storage: 3.5TB\1024\0.023 \= \$82 per month <br><br> Shards: 500GB \== 5.8MB/s \== 6 6\24\30\0.015 \= \$65 <br><br> Monthly estimate: \~ \$3200 |

|             |                                         |                                         |                                         |                                         |
|-------------|-----------------------------------------|-----------------------------------------|-----------------------------------------|-----------------------------------------|
| Dedicated   | (same as above)                         | (same as above)                         | (same as above)                         | (same as above)                         |
| SM on-cloud | (same as above)                         | (same as above)                         | (same as above)                         | (same as above)                         |
| SM on-prem  | Estimated with reference architectures. | Estimated with reference architectures. | Estimated with reference architectures. | Estimated with reference architectures. |

Cost analysis with respect to GitLab reference architectures

Read for more details: <https://docs.gitlab.com/administration/reference\architectures/>

Before estimating resources, we'll need a measure of message traffic across the different reference architectures:

- Message volume: Estimating how many messages per second.
- Message size: Estimating message payload size in bytes.
- Number of publishers/consumers
- Persistence/storage: How much data needs to be stored and/or retained over how much time.

| Reference Architecture / System | Kafka | RabbitMQ | PubSub | Kinesis | NATS |

| ---- | ---- | ---- | ---- | ---- | ---- |

| General overview, sizing constraints. | When compared to NATS or RabbitMQ, Kafka can be more resource intensive both in compute and storage given its replication overheads. It'll consume more compute nodes if we use Zookeeper for cluster coordination OR more vCPUs in the case of using KRaft which works on the same JVM as the broker process. | RabbitMQ can be slightly more compute intensive as compared to NATS given its acknowledgement mechanism, wherein it's more suitable for more transactional streaming workloads. | Estimated costs: <br><br> Message ingestion: \$40 per TB (\$0.04/GB) <br><br> Message delivery: \$40 per TB <br><br> Storage retention: \$0.27/GB/month <br><br> Data transfer: Standard GCP egress fees. | Estimated costs: <br><br> Shards (compute): \$0.015 per shard/hour <br><br> Ingestion (writes): \$0.036 per million records <br><br> Data retention: \$0.02 per GB per day <br><br> Data transfer: Standard AWS egress fees. | A simple 3-nodes cluster should suffice, instance-sizing to be estimated with traffic estimates. |

| Small (~2000 users) | | Roughly similar to NATS. | Sizing: 5000 messages per second.

<br><br> Ingestion cost: \$520 <br><br> Delivery cost: \$520 <br><br> Storage cost: \$2700

<br><br> Monthly estimate: \$3700 at the minimum. | Sizing: 5000 messages per second, needing 3

shards. <br><br> Shard cost: \$32.40 <br><br> Ingestion cost: \$3.60 <br><br> Storage cost: \$6000

<br><br> Monthly estimate: \$6000 at the minimum | Sizing: 4vCPU, 8GB RAM, 100GB SSD, 1TB total data transfer. <br><br> Instances: 3 |

| | | | | AWS t3.xlarge @ \$0.16/hr \= \$120 per month. <br><br> 100 GB EBS gp3 SSD \= \$10 per month. <br><br> Bandwidth: 1TB @ \$0.09GB \= \$90 <br><br> Monthly estimate: \$480 at the minimum. |

| | | | | GCP e2-standard-4 \= \$108 per month. <br><br> Persistent SSD (100GB) \= \~\$17 per month. <br><br> Bandwidth: 1TB @ \$0.08GB \= \$80 <br><br> Monthly estimate: \$450 at the minimum. |

| Medium (~5000 users) | | Roughly similar to NATS. | Sizing: 10000 messages per second.

<br><br> Ingestion cost: \$1040 <br><br> Delivery cost: \$1040 <br><br> Storage cost: \$2700

<br><br> Monthly estimate: \$4700 at the minimum. | Sizing: 10000 messages per second, needing 5

shards. <br><br> Shard cost: \$54 <br><br> Ingestion cost: \$7.20 <br><br> Storage cost: \$6000

<br><br> Monthly estimate: \$6000 at the minimum | Sizing: 8vCPU, 16GB RAM, 100GB SSD, 2TB total data transfer. <br><br> Instances: 3 |

| | | | | AWS m6i.2xlarge @ \$0.38/hr \= \$280 per month. <br><br> 100 GB EBS gp3 SSD \= \$10 per month. <br><br> Bandwidth: 2TB @ \$0.09GB \= \$180 <br><br> Monthly estimate: \$1050 |

| | | | | GCP n2-standard-8 \= \$240 per month. <br><br> Persistent SSD (100GB) \= \~\$17 per month <br><br> Bandwidth: 2TB @ \$0.08GB \= \$160 <br><br> Monthly estimate: \$940 at the minimum. |

| Large (~10000 users) | | Roughly similar to NATS. | Sizing: 50000 messages per second.

<br><br> Ingestion cost: \$5200 <br><br> Delivery cost: \$5200 <br><br> Storage cost: \$2700

<br><br> Monthly estimate: \$13000 at the minimum. | Sizing: 50000 messages per second, needing

10 shards. <br><br> Shard cost: \$110 <br><br> Ingestion cost: \$36 <br><br> Storage cost: \$6000

<br><br> Monthly estimate: \$6200 at the minimum | Sizing: 8vCPU, 32GB RAM, 500GB SSD, 5TB total

data transfer. <br><br> Instances: 5 |  
 | | | | | AWS m6i.4xlarge @ \$0.76/hr \= \$560 per month. <br><br> 500 GB EBS gp3 SSD \= \$50 per month. <br><br> Bandwidth: 5TB @ \$0.09GB \= \$450 <br><br> Monthly estimate: \$3500 |  
 | | | | | GCP n2-standard-16 \= \$480 per month. <br><br> Persistent SSD (500GB) \= \~\$85 per month. <br><br> Bandwidth 5TB @ \$0.08GB \= \$400 <br><br> Monthly estimate: \$3225 at the minimum. |  
 | Extra Large (\~25000 users) | | Roughly similar to NATS. | Sizing: 250000 messages per second. <br><br> Ingestion cost: \$25000 <br><br> Delivery cost: \$25000 <br><br> Storage cost: \$2700 <br><br> Monthly estimate: \$52700 at the minimum. | Sizing: 250,000 messages per second, needing 25 shards. <br><br> Shard cost: \$500 <br><br> Ingestion cost: \$360 <br><br> Storage cost: \$6000 <br><br> Monthly estimate: \$7000 at the minimum | Sizing: 16vCPU, 64GB RAM, 1TB SSD, 10TB total data transfer <br><br> Instances: 5+ |  
 | | | | | AWS c6i.8xlarge @ \$1.53/hr \= \$1120 per month. <br><br> 1TB EBS gp3 SSD \= \$100 per month. <br><br> Bandwidth: 10TB @ \$0.09GB \= \$900 <br><br> Monthly estimate: \$7000 at the minimum. |  
 | | | | | GCP n2-highcpu-32 \= \$1000 per month. <br><br> Persistent SSD (1TB) \= \~\$130 per month. <br><br> Bandwidth: 10TB @ \$0.08GB \= \$800 <br><br> Monthly estimate: \$6450 at the minimum. |

## Conclusions from a features & cost perspective

- A significant factor worth noting from analysing these backends is their high operational, distribution & support overheads both for running them ourselves OR expecting them to be available inside non-hosted environments such as Self-Managed.
- All things considered, NATS appears to be the least expensive across all backends, considering it ships as a single Go-binary and can be installed in close-proximity to user services/applications with zero external dependencies. When needed, it can be scaled/sharded out across multiple servers/clusters subject to which reference architecture we run it within.
- The only factor favoring managed cloud-solutions instead is any support costs involved but their operational costs seem to outweigh any benefits we derive from using them, especially as their usage and/or adoption grows with our scale.

The rest of this document focuses on building out the proposal to use NATS as the messaging solution within a GitLab instance.

## Components

- One or more JetStream-enabled NATS servers
- Persistent volumes, preferably SSDs, for each deployed NATS server.
- Decentralized authentication callout server (long-term only) : For authentication/authorisation, we aim to start with leveraging hard-coded roles & credentials within NATS (centralized) but in the long-term, we expect to use a decentralized server-side auth-callout implementation for authenticating inbound traffic.

## Deployments

With the intention to make NATS available to every GitLab installation, we aim to build necessary support for the following deployment models:

| Deployment type | Proposed topology |

|---|---|  
| GDK | Running local to the installation |  
| .com | One or more dedicated cluster(s) |  
| Cells | One or more clusters subject to cells-topology |  
| Dedicated | One dedicated cluster per instance |  
| Self-Managed | Standalone cluster subject to distribution/instance-sizing |

## Tenancy

For all deployment-types, we expect a given NATS cluster to be multi-tenant, i.e. multiple users/services hosted on the deployment to share the same underlying NATS infrastructure for their messaging needs.

## Rollout roadmap

| Deliverables | Timeline |  
|---|---|  
| Pre-staging/Testing | FY26Q1 |  
| Staging | TBD |  
| Production: .com | TBD |  
| Gradual rollout to Dedicated | TBD |  
| Gradual rollout to Self-managed | TBD |

## Design & Implementation

### Architecture

The following diagram depicts a simple 3-node NATS cluster deployed as a StatefulSet with dedicated persistent volumes. A given application can then discover the service:

- using a headless service, or

text

nats.default.svc.cluster.local

- addressing the pods directly

text

nats-0.nats.default.svc.cluster.local

nats-1.nats.default.svc.cluster.local

nats-2.nats.default.svc.cluster.local

## Setup

- Setup an N-nodes cluster subject to data volumes & retention.
- Enable data persistence via NATS JetStream.



- Configure stream replication for subjects as needed.

## Connectivity

- NATS comes with support for both plaintext and TLS connections. We intend to use TLS-enabled connections to help authenticate all incoming traffic.
- Access to the NATS service will only be available to internal network clients. External clients will not be able to connect to the NATS service.
- External load-balancing is therefore not needed, which is also highly advised against. Each NATS server in a given cluster must be reachable individually with all loadbalancing deferred to NATS itself.
- For Cells-based installations, we can leverage the use of NATS Gateways to connect clusters should we need to, but we expect each cell to be serviceable in isolation for majority of our applications. While we believe that NATS Gateways may help with interconnection in future, this functionality is out-of-scope for this iteration of the blueprint.

## Topology

- For all deployment-types, NATS will be deployed region-local only similar to Redis or Sidekiq and not Geo-replicated like Postgres. While it is possible to mesh NATS clusters together as a feature, this functionality is out-of-scope for this iteration of the blueprint.
- For GitLab.com, we intend to setup & run NATS clusters cloud-natively on Kubernetes with the assumption that the following potential overheads can be well-managed:
  - running stateful workloads within Kubernetes.
  - performance overheads from routing traffic within Kubernetes services.
  - load-balancing traffic via Kubernetes service-topology.
- From an operational perspective and considering we prefer new services to be built Kubernetes-based, it'll also be trivial for us to use the same configurations for GitLab.com, Dedicated, Cells and Self-Managed going forward.
- If running it cloud-natively does incur overheads, we can resort to running NATS directly on VMs within the same VPC/network-boundaries to leverage better utilization of the underlying hardware and reduce any operational complexity.
- We have also prototyped both of these deployment models:
  - using a Helm chart for cloud native installations, initial POC
  - using Terraform to install NATS on cloud VMs directly - initial POC

## Data persistence

- NATS allows all ingested data to be stored in-memory or persisted on-disk.
- We intend to leverage NATS JetStream to persist all ingested data durably.
- We expect to use SSDs to improve the performance of all read/write operations.
- We intend to enforce retention policies for ingested data and provision underlying storage with sufficient headroom to begin with especially as we tune our retention needs gradually.

From an operational perspective, it is necessary we have the ability to increase underlying storage trivially when needed.

- We expect to enable data compression to reduce storage footprint further.

### Integration with GitLab

- We can add the aforementioned NATS connection string to the instance configuration to be used by services/applications. For example:

- Add `nats['address'] = 'nats://nats.default.svc.cluster.local:4222'` to `/etc/gitlab/gitlab.rb`, or
- Override `NATSURL=nats://nats.default.svc.cluster.local:4222` in `gitlab.yml`.

### Security

#### Authentication/Authorization

- NATS has prebuilt support for connections encrypted over TLS.
- It also comes with centralized auth support via JWT/NKEYS.
- We expect to promote the usage of separate principals (users/accounts) across distinct systems, e.g. producers/consumers of a given stream.
- NATS offers grouping of clients and subject space with accounts.

#### Example authentication/authorization setup

We take Siphon as an example use-case here, wherein we cover:

- Creation of accounts to isolate clients.
- Adding users to these accounts with specific permissions for available subjects.

Authentication will be achieved via nkeys.

- Producers and consumers have their own users and permissions.
- Producer/Consumer nkeys are to be treated with the same security practices as we currently do for our database secrets.

#### Example authorization.conf

text

listen: 127.0.0.1:4222

jetstream: enabled

```
producerpermissions = {  
  publish = ">"  
  subscribe = ">"  
}
```

```
consumerpermissions = {  
  publish = {  
    deny = ">"  
  }  
  subscribe = ">"  
}
```

```

accounts: {
  siphon: {
    users: [
      {nkey: Uxx, permissions: $producerpermissions},
      {nkey: Uxx, permissions: $consumerpermissions}
    ]
  }
}

```

In the above configuration, producer is allowed to publish and subscribe over all the subject space for siphon account while consumer is only allowed to subscribe to available subjects.

We can also apply further granularity on subject space if desired. The use of permissions map allows for such fine-grained control.

Example usage of this configuration:

```

go
func TestServerConfiguration(t testing.T) {
  server, := RunServerWithConfig("authorization.conf")
  t.Logf(server.ClientURL())

  // producernkey.txt holds the nkey seed
  opt, err := nats.NkeyOptionFromSeed("producernkey.txt")
  if err != nil {
    t.Error(err)
    return
  }
  nc, err := nats.Connect(server.ClientURL(), opt)
  if err != nil {
    t.Error(err)
    return
  }
  js, err := nc.JetStream()
  if err != nil {
    t.Error(err)
    return
  }
  = js.Streams()
  defer nc.Close()
}

```

## Encryption

- While NATS has support for encryption at rest, it recommends the use of filesystem encryption when available.
- Considering our deployment-targets, we intend to use disk-encryption - available on both GCP

and AWS with support for additional customer-managed key options.

### Auditing/Logging

- We intend to ship necessary NATS logs to our centralized logging infrastructure to enable any auditing/monitoring purposes.

### Operations

### Scalability

NATS is extremely lightweight and can support ingesting & digesting high amounts of messages with sub-millisecond latencies. Given its architecture, it's also optimized for handling backpressure and exercise flow-control subject to traffic volumes.

We ran the following preliminary tests against a Kubernetes-based NATS cluster with 3 servers, each such server (pod) running on a c2d-standard-16 GKE node and attached to a 100GB pd-balanced SSD persistent volume for data storage.

Note, the underlying GKE cluster is a regional cluster with cluster-nodes spread in 3 distinct AZs. NATS servers (pods) were carefully spread across the 3 AZs at all times during our tests.

### Key insights

- CPU usage is directly proportional to cluster usage with large spikes in the case of asynchronous publishers producing a very large number of events in a short period of time.
- Memory usage remained stable regardless of the amount of events ingested.
- Cross-AZ replication does affect cluster throughput but overall performance remains well-beyond our immediate needs.

### Synchronous publisher with stream replication

CPU usage remained consistently low while write-throughput takes a notable hit.

text

```
platform-pre-stg kubectl -n nats exec -it nats-box-6888bbc55c-kd6tm -- nats
--server=nats://nats.nats.svc.cluster.local:4222 bench foobar --pub 1 --sub 5 --msgs=1000000
--js --maxbytes 20GB --purge --replicas=2 --syncpub
10:55:59 JetStream ephemeral ordered push consumer mode, subscribers will not acknowledge the
consumption of messages
10:55:59 Starting JetStream benchmark [subject=foobar, multisubject=false,
multisubjectmax=100000, js=true, msgs=1,000,000, msgsize=128 B, pubs=1, subs=5,
stream=benchstream, maxbytes=20 GiB, storage=file, syncpub=true, pubbatch=100, jstimeout=30s,
pull=false, consumerbatch=100, push=false, consumername=natscli-bench, replicas=2, purge=true,
pubsleep=0s, subsleep=0s, dedup=false, dedupwindow=2m0s]
```

NATS Pub/Sub stats: 7,957 msgs/sec ~ 994.75 KB/sec

Pub stats: 1,326 msgs/sec ~ 165.81 KB/sec

Sub stats: 6,631 msgs/sec ~ 828.96 KB/sec

[1] 1,326 msgs/sec ~ 165.80 KB/sec (1000000 msgs)

[2] 1,326 msgs/sec ~ 165.80 KB/sec (1000000 msgs)  
[3] 1,326 msgs/sec ~ 165.80 KB/sec (1000000 msgs)  
[4] 1,326 msgs/sec ~ 165.79 KB/sec (1000000 msgs)  
[5] 1,326 msgs/sec ~ 165.81 KB/sec (1000000 msgs)  
min 1,326 | avg 1,326 | max 1,326 | stddev 0 msgs

Asynchronous publisher with stream replication, publishing batches in sizes 100, 1000, 10000

CPU usage is proportional to batch-size with write-throughput improving with moderately sized batches.

text

```
· platform-pre-stg kubectl -n nats exec -it nats-box-6888bbc55c-kd6tm -- nats
--server=nats://nats.nats.svc.cluster.local:4222 bench foobar --pub 1 --sub 5 --msgs=1000000
--js --maxbytes 20GB --purge --replicas=2
11:13:02 JetStream ephemeral ordered push consumer mode, subscribers will not acknowledge the
consumption of messages
11:13:02 Starting JetStream benchmark [subject=foobar, multisubject=false,
multisubjectmax=100000, js=true, msgs=1,000,000, msgsize=128 B, pubs=1, subs=5,
stream=benchstream, maxbytes=20 GiB, storage=file, syncpub=false, pubbatch=100, jstimeout=30s,
pull=false, consumerbatch=100, push=false, consumername=natscli-bench, replicas=2, purge=true,
pubsleep=0s, subsleep=0s, dedup=false, dedupwindow=2m0s]
```

NATS Pub/Sub stats: 274,880 msgs/sec ~ 33.55 MB/sec

Pub stats: 46,073 msgs/sec ~ 5.62 MB/sec

Sub stats: 229,066 msgs/sec ~ 27.96 MB/sec

[1] 46,021 msgs/sec ~ 5.62 MB/sec (1000000 msgs)  
[2] 45,866 msgs/sec ~ 5.60 MB/sec (1000000 msgs)  
[3] 45,901 msgs/sec ~ 5.60 MB/sec (1000000 msgs)  
[4] 45,813 msgs/sec ~ 5.59 MB/sec (1000000 msgs)  
[5] 45,986 msgs/sec ~ 5.61 MB/sec (1000000 msgs)  
min 45,813 | avg 45,917 | max 46,021 | stddev 76 msgs

```
· platform-pre-stg kubectl -n nats exec -it nats-box-6888bbc55c-kd6tm -- nats
--server=nats://nats.nats.svc.cluster.local:4222 bench foobar --pub 1 --sub 5 --msgs=1000000
--js --maxbytes 20GB --purge --replicas=2 --no-progress --pubbatch=1000
11:17:38 JetStream ephemeral ordered push consumer mode, subscribers will not acknowledge the
consumption of messages
11:17:38 Starting JetStream benchmark [subject=foobar, multisubject=false,
multisubjectmax=100000, js=true, msgs=1,000,000, msgsize=128 B, pubs=1, subs=5,
stream=benchstream, maxbytes=20 GiB, storage=file, syncpub=false, pubbatch=1,000,
jstimeout=30s, pull=false, consumerbatch=100, push=false, consumername=natscli-bench,
replicas=2, purge=true, pubsleep=0s, subsleep=0s, dedup=false, dedupwindow=2m0s]
```

NATS Pub/Sub stats: 524,251 msgs/sec ~ 64.00 MB/sec

Pub stats: 152,262 msgs/sec ~ 18.59 MB/sec

Sub stats: 436,876 msgs/sec ~ 53.33 MB/sec

[1] 100,985 msgs/sec ~ 12.33 MB/sec (1000000 msgs)  
[2] 88,393 msgs/sec ~ 10.79 MB/sec (1000000 msgs)

[3] 88,367 msgs/sec ~ 10.79 MB/sec (1000000 msgs)  
[4] 87,896 msgs/sec ~ 10.73 MB/sec (1000000 msgs)  
[5] 87,375 msgs/sec ~ 10.67 MB/sec (1000000 msgs)  
min 87,375 | avg 90,603 | max 100,985 | stddev 5,204 msgs

· platform-pre-stg kubectl -n nats exec -it nats-box-6888bbc55c-kd6tm -- nats  
--server=nats://nats.nats.svc.cluster.local:4222 bench foobar --pub 1 --sub 5 --msgs=1000000  
--js --maxbytes 20GB --purge --replicas=2 --no-progress --pubbatch=10000  
11:17:57 JetStream ephemeral ordered push consumer mode, subscribers will not acknowledge the consumption of messages  
11:17:57 Starting JetStream benchmark [subject=foobar, multisubject=false, multisubjectmax=100000, js=true, msgs=1,000,000, msgsize=128 B, pubs=1, subs=5, stream=benchstream, maxbytes=20 GiB, storage=file, syncpub=false, pubbatch=10,000, jstimeout=30s, pull=false, consumerbatch=100, push=false, consumername=natscli-bench, replicas=2, purge=true, pubsleep=0s, subsleep=0s, dedup=false, dedupwindow=2m0s]

NATS Pub/Sub stats: 424,064 msgs/sec ~ 51.77 MB/sec  
Pub stats: 70,985 msgs/sec ~ 8.67 MB/sec  
Sub stats: 353,386 msgs/sec ~ 43.14 MB/sec  
[1] 71,156 msgs/sec ~ 8.69 MB/sec (1000000 msgs)  
[2] 70,899 msgs/sec ~ 8.65 MB/sec (1000000 msgs)  
[3] 70,757 msgs/sec ~ 8.64 MB/sec (1000000 msgs)  
[4] 70,887 msgs/sec ~ 8.65 MB/sec (1000000 msgs)  
[5] 70,812 msgs/sec ~ 8.64 MB/sec (1000000 msgs)  
min 70,757 | avg 70,902 | max 71,156 | stddev 137 msgs

Asynchronous publisher, testing pull vs push consumers

Nothing noteworthy about CPU usage with pull consumers performing better than push ones.

text

· platform-pre-stg kubectl -n nats exec -it nats-box-6888bbc55c-kd6tm -- nats  
--server=nats://nats.nats.svc.cluster.local:4222 bench foobar --pub 1 --sub 5 --msgs=1000000  
--js --maxbytes 20GB --purge --replicas=2 --no-progress --pubbatch=100 --push  
11:24:53 JetStream durable push consumer mode, subscriber(s) will explicitly acknowledge the consumption of messages  
11:24:53 JetStream ephemeral ordered push consumer mode, subscribers will not acknowledge the consumption of messages  
11:24:53 Starting JetStream benchmark [subject=foobar, multisubject=false, multisubjectmax=100000, js=true, msgs=1,000,000, msgsize=128 B, pubs=1, subs=5, stream=benchstream, maxbytes=20 GiB, storage=file, syncpub=false, pubbatch=100, jstimeout=30s, pull=false, consumerbatch=100, push=true, consumername=natscli-bench, replicas=2, purge=true, pubsleep=0s, subsleep=0s, dedup=false, dedupwindow=2m0s]

NATS Pub/Sub stats: 65,697 msgs/sec ~ 8.02 MB/sec  
Pub stats: 32,912 msgs/sec ~ 4.02 MB/sec  
Sub stats: 32,848 msgs/sec ~ 4.01 MB/sec  
[1] 6,581 msgs/sec ~ 822.69 KB/sec (200000 msgs)  
[2] 6,586 msgs/sec ~ 823.28 KB/sec (200000 msgs)

[3] 6,574 msgs/sec ~ 821.78 KB/sec (200000 msgs)  
[4] 6,575 msgs/sec ~ 821.90 KB/sec (200000 msgs)  
[5] 6,579 msgs/sec ~ 822.48 KB/sec (200000 msgs)  
min 6,574 | avg 6,579 | max 6,586 | stddev 4 msgs

· platform-pre-stg kubectl -n nats exec -it nats-box-6888bbc55c-kd6tm -- nats  
--server=nats://nats.nats.svc.cluster.local:4222 bench foobar --pub 1 --sub 5 --msgs=1000000  
--js --maxbytes 20GB --purge --replicas=2 --no-progress --pubbatch=100 --pull  
11:25:56 JetStream durable pull consumer mode, subscriber(s) will explicitly acknowledge the  
consumption of messages  
11:25:56 Starting JetStream benchmark [subject=foobar, multisubject=false,  
multisubjectmax=100000, js=true, msgs=1,000,000, msgsize=128 B, pubs=1, subs=5,  
stream=benchstream, maxbytes=20 GiB, storage=file, syncpub=false, pubbatch=100, jstimeout=30s,  
pull=true, consumerbatch=100, push=false, consumername=natscli-bench, replicas=2, purge=true,  
pubsleep=0s, subsleep=0s, dedup=false, dedupwindow=2m0s]

NATS Pub/Sub stats: 95,057 msgs/sec ~ 11.60 MB/sec  
Pub stats: 47,747 msgs/sec ~ 5.83 MB/sec  
Sub stats: 47,528 msgs/sec ~ 5.80 MB/sec  
[1] 15,410 msgs/sec ~ 1.88 MB/sec (200000 msgs)  
[2] 12,056 msgs/sec ~ 1.47 MB/sec (200000 msgs)  
[3] 11,976 msgs/sec ~ 1.46 MB/sec (200000 msgs)  
[4] 9,556 msgs/sec ~ 1.17 MB/sec (200000 msgs)  
[5] 9,530 msgs/sec ~ 1.16 MB/sec (200000 msgs)  
min 9,530 | avg 11,705 | max 15,410 | stddev 2,157 msgs

## Monitoring

- Inbuilt monitoring exposed as Prometheus metrics, details [here](#).

## Cluster upgrades

- NATS provides well-documented paths around upgrading clusters, as long as we provision multi-node clusters and can perform a gradual rolling-restart.

- In case of running NATS via Statefulsets on Kubernetes, this can be handled automatically by the underlying Kubernetes statefulset controller considering we use `.spec.updateStrategy` to be `RollingUpdate` which ensures an incremental rollout of the updates one pod at a time.

- In case of running NATS on bare VMs, an operator will have to perform such a rolling upgrade of the cluster either manually or via tooling depending on how the cluster is setup.

## Failure Scenarios

- In the event of total service unavailability of NATS, we might experience loss of data especially in scenarios where NATS is where we land incoming data first.

- In the event of loss of one or more nodes in a given NATS cluster, clients can still continue to push new events as long as other stream-replicas are configured and can assume new

leadership for affected streams. For clients that cannot withstand loss of messages, stream replication is highly advised which ensures any affected streams will recover as soon as any cluster-deterioration is remedied.

- While stream replication ensures data redundancy for ingested data, asynchronous writes might still lead to loss of data. To minimize any loss of data, clients should prefer synchronous writes to ensure all ingested data is durably replicated before their writes get acknowledged, with the caveat that synchronous writes will reduce overall write-performance.

Note Further details of how NATS publishers or consumers must be designed is out of scope for this blueprint. All user-facing documentation for building NATS applications will be developed separately.

- All ingested data is persisted durably via NATS JetStream. In the event of unrecoverable messages however, we can rely on an explicit disaster recovery setup to recover data, which includes:

- Automatic recovery in case of intact quorum nodes for replicated streams, or
- Manual recovery from periodic stream backups.

Note To ensure all persisted data is recoverable, we'll need to integrate such a disaster recovery procedure for NATS into our general GitLab DR plans & tooling, taking into consideration all GitLab deployment types.

Note, in the specific case of Siphon, all data buffered within NATS and due to be exported to ClickHouse also remains available in Postgres. In the scenario where Siphon fails to connect to NATS or there is data loss on NATS, Siphon can perform a full-resync to ensure data consistency across Postgres & ClickHouse again. For other use-cases where this is not possible, we'll have to depend on recovering lo